With this code in place, the controller test and the integration test pass.

You're all green.

Except you're not.

The add_task_spec.rb test, which displays a project show page, now redirects. You created a user but didn't add the user to the project. You need to make a slight change to the setup—you must create a Role associating the user and the project in order for the code to let you view the project page:

security/02/spec/system/add_task_spec.rb

```
let(:project) { create(:project, name: "Project Bluebook") }
let!(:task_1) { create(
  :task, project: project, title: "Search Sky", size: 1, project_order: 1) }
let!(:task_2) { create(
  :task, project: project, title: "Use Telescope", size: 1,
         project_order: 2) }
let!(:task_3) { create(
  :task, project: project, title: "Take Notes", size: 1, project_order: 3) }
let(:user) { create(:user) }

before(:example) do
  Role.create(user: user, project: project)
  sign_in(user)
end
```

Now you're green.

More Access-Control Testing

The advantage of splitting responsibility and testing into separate controller and model concerns becomes even more clear when you add another requirement. Let's allow for the possibility of administrative users who can see any project, as well as public projects that can be seen by any user.

You'll want to represent these properties in the database—in this case, you're doing the design work based on planning your test. You'll generate a migration using the command rake generate migration add_public_fields, which gives a skeleton file. Add the following and then run rake db:migrate:

```
$ rails generate migration add_public_fields
```

security/03/db/migrate/20170904151009_add_public_fields.rb

```
class AddPublicFields < ActiveRecord::Migration[5.1]
  def change
    change_table :projects do |t|
      t.boolean :public, default: false
    end
```